2) Configuring and securing SSH

- What is SSH ?
- What is OpenSSH ?
  - What OpenSSH provide ?
  - What OpenSSH is used for ?
- Using command line with SSH
  - Identifying remote users
  - Making .ssh file for new users
- SSH host keys
  - What inside known_host file:
- SSH key-based authentication
  - 1) Generating SSH keys
    - SSH key lookup order
  - 2) Sharing the public key
- Customizing OpenSSH service configuration
  - Prohibit access to the root
  - Prohibit using password-based auth
- Resources

What is SSH ?

- SSH itself is just a **protocol (set of rules)**, it's an **application protocol** that describes how secure communication should work.
- It is an **application-layer protocol** that runs over TCP and provides secure remote access and encrypted communication services.
- It defines:
  1. Key exchange
  2. Encryption
  3. Authentication
  4. Message formats
  5. Session management.
- All on top of TCP (port 22)

> ⓘ **SSH itself is not software, it's a standard protocol.**

```
TCP/IP model

Application   -> SSH, HTTP, FTP, rsync
Transport     -> TCP
Internet      -> IP
Link          -> Ethernet / Wi-Fi
```

```
OSI model:

Layer 7 — Application   -> SSH
Layer 6 — Presentation -> Encryption, compression (handled by SSH)
Layer 5 — Session      -> Session management (handled by SSH)
Layer 4 — Transport    -> TCP
Layer 3 — Network      -> IP
```

- So in short, SSH doesn't:
  1. Creates a new network layer.
  2. It does not replace TCP/IP.
- Example using rsync:

```
rsync (application) -> SSH (application — encryption & auth) -> TCP
(reliable transport) -> IP (routing)
```

What is OpenSSH ?

- It's an open-source implementation of the SSH (secure shell) protocol (actual code), which allows us to securely connect and communicate to a remote computers over an **insecure network**.
- Think of OpenSSH as something that acts on those rules which they are the SSH rules themselves.
- Written mainly in C, by OpenBSD team.
- **On a public network, SSH creates an encrypted connection (tunnel) that allows data to be sent and received securely.**

Concept	Example
Protocol	Traffic laws
Implementation	Cars that follow those laws

What OpenSSH provide ?

Program	What it is
`ssh`	Client program
`sshd`	Server daemon
`scp`	Secure copy tool
`sftp`	Secure file transfer
`ssh-keygen`	Key generator
`ssh-agent`	Key manager
`ssh-copy-id`	Install public key on server

What OpenSSH is used for ?

1. Secure remote login:

```
ssh user@server_ip
```

- It's a replacement for telnet or rlogin.

> ⓘ **telnet and Rlogin are older network protocols for logging into and controlling remote computers, acting as a text-based "window" to another machine, but both are highly insecure, sending data (including passwords) in plaintext.**

2. Secure file transfer:

```
scp file.txt user@server:/path/
```

or sync directories:

```
rsync -avz foler/ user@server:/path/
```

> ⓘ **-a is short for archive, means sync the directories as they are, -v verbose (details), -z to compress the dir.**

3. Key-based authentication (passwordless login)

```
ssh-keygen
ssh-copy-id user@server
```

4. Port forwarding (SSH tunneling)

Expose or protect services securely:

```
ssh -L 8080:localhost:80 user@server
```

Very useful for:

- Databases
- Web apps
- Remote debugging

Using command line with SSH

- We can ssh using:

```
ssh remotehost
```

- Now the remote-host should be the IP address of that host.
- host-names in general they are a human readable texts, but when we want to connect, the network needs a numeric address (IP) to know where to send packets.

```
ssh username@remote-host
```

- Now if we want to connect to a specific username on that remote host, we can use the above command.

```
ssh username@remote-host ls #THis will list files when you enter the
password
```

- Also we can send commands directly when connecting throw ssh (it needs password approval).

> ⓘ **Solving problems on overTheWire is useful to understand more**

Identifying remote users

- The `w` command is very important which used to displays a list of users currently logged into the computer (either physical or throw ssh).
- It shows:
  1. user name
  2. TTY (pusedo terminal)
  3. FROM (IP address if it was ssh)
  4. LOGIN@ (what time)
  5. IDLE (you didn't run any commands for some times)
  6. What --> shows the command or activity the user currently running.

Making .ssh file for new users

- When making a new `.ssh` file for new users, we must do 3 things:
  1. chmod the .ssh directory into 700
  2. Making `authorized_keys` file in `~/.ssh/` and `chmod 600` for it.
  3. `chown` for `~/.ssh/authorized_file` into the user who made it.

> **`authorized_keys` tells the SSH server which public keys are allowed to log into a specific user account. So when you do `ssh-copy-id`, the public key copied to this file.**

SSH host keys

- What happened when a user uses SSH to connect to a server?
  - The `ssh` command checks to see if it has a copy of the public key for that server it tries to connect to in: `/etc/ssh/ssh_known_hosts(sysetm wide)` or `~/.ssh/known_hosts (per user)`
  - If the server's key exists but changes, SSH will refuse to connect:
    - You may ask, how it will knows that the public host key, changed ?
      - This is because the `known_host` file saves in this format `hostname/ IP --> public host key`.
      - So it knows that this hostname "example.com" --> have this IP, so if it changed means something wrong and we shouldn't connect to it until changed manually from the known_host file.
  - If it didn't found it, it will ask your confirmation to continue the connection, and the server public key will be saved in the known_host file.

What inside `known_host` file:

hostname

```
[user01@host ~]$ cat ~/.ssh/known_hosts
remotehost,172.25.250.11 ecdsa-sha2-nistp256
AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBOsEi0e+FlaNT6jul8Ag5Nj
+RViZl0yE2w6iYUr+1fPtOIF0EaOgFZ1LXM37VFTxdgFxHS3D5WhnIfb+68zf8+w=
```

Algo IP

- First field: hostname --> sometimes you will find it hashed for security reasons.
- Second: encryption algorithm for the key
- Third: The key itself.

> **Each remote SSH server that you connect to stores the public key in `/etc/ssh/` or in `~/.ssh/` with the extension `.pub`**

SSH key-based authentication

- Now we can configure a server to authenticate without a password and just with the public-private key.
- The public key is given to the server and the private key is saved in the place where you want to access that server, and it must kept secure (The private key).

1) Generating SSH keys

- We can use `ssh-keygen` command to make a private and public key, private keys are saved as `~/.ssh/id_rsa`, and public as `~/.ssh/id_rsa.pub` by default (until you change there name).
- There is a very important concept here, what happened when you connect using SSH:
  1. The private file is encrypted, so SSH cannot use it unless it decrypts it.
  2. To decrypt it we need to use the passphrase (password).
  3. Something must provide the passphrase:
     a) Now you can't store the private key unencrypted -> this is bad and your identity may be stolen.
     b) Store the passphrase somewhere, (bad) you can't just save the passphrase anywhere.
     c) Give it (save it) in the memory using the `ssh-agent`:
     - It's a background process that:
       1. Holds decrypted private keys in memory (RAM) --> Caching it in memory
       2. Never writes them to disk.
     - You unlock your key once (write the passphrase) using `ssh-add` like `ssh-add ~/.ssh/id_rsa`
       After that:
       - The agent keeps the decrypted key in RAM

- SSH can use it **without asking for the passphrase again**

```
[user@host ~]$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/user/.ssh/id_rsa): Enter
Created directory '/home/user/.ssh'.
Enter passphrase (empty for no passphrase): Enter
Enter same passphrase again: Enter
Your identification has been saved in /home/user/.ssh/id_rsa.
Your public key has been saved in /home/user/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:vxutUNPio3QDCyvkYm1oIx35hmMrHpPKWFdIYu3HV+w user@host.lab.example.com
The key's randomart image is:
+---[RSA 2048]----+
|                 |
|   .     .       |
|  o o     o      |
| . = o   o .     |
|  o + = S E .    |
| ..O o + * +     |
|.+% O . + B .    |
|=*oO . . + *     |
|++.     . +.     |
```

Directory

Password

Confirm

> **if the `ssh-agent` didn't work on your system automatically (by the daemon), you can run it using `eval $(ssh-agent)` -> it will give you `Agent pid ****`**

> **The `eval` command set automatically the variables asked by the agent so you don't have to, it also displays the PID of the `ssh-agen` process.**

SSH key lookup order

When you run:

```
ssh user@host
```

1. SSH first loads configuration from:

- `/etc/ssh/ssh_config`
- `~/.ssh/config`

2. **Ask `ssh-agent`**:
   If an **ssh-agent** is running and SSH is allowed to use it, SSH will:

   1. Ask the agent for available keys.
   2. Try those keys with the server.

Important details:

- The **private keys stay inside the agent**
- SSH only asks the agent to **sign a challenge**
- The keys are already **decrypted in memory**

If not found in the agent:
- SSH searches for private key files in:

```
~/.ssh/id_rsa
~/.ssh/id_ecdsa
~/.ssh/id_ecdsa_sk
~/.ssh/id_ed25519
~/.ssh/id_ed25519_sk
~/.ssh/id_dsa (deprecated)
```

- If a key is found:
  - SSH asks you for the **passphrase**
  - Decrypts the key
  - Uses it for authentication
- **If nothing works**:
  - SSH falls back to password authentication (if allowed)

3. SSH may try keys from `IdentityFile`

If `~/.ssh/config` contains:

```
Host github.com
    IdentityFile ~/.ssh/github_key
```

Then SSH will **try that key before default keys**.

> ⓘ **If you don't have your private key in the ssh-agent, you may get a message says Make sure you have the right authentication . This is because sometimes ssh doesn't check the disk at all. --> Make sure you always have your `ssh-agent` running to solve this.**

2) Sharing the public key

- We can share the public key with the meaning server by using `ssh-copy-id` command:

```
ssh-copy-id -i <Public-key_file_path> user@remote-host
```

```
[user@host ~]$ ssh-copy-id -i .ssh/key-with-pass.pub user@remotehost
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/user/.ssh/
id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
 out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted
 now it is to install the new keys
user@remotehost's password: redhat
Number of key(s) added: 1
```

- `-i` to identify the file named `.ssh/key...`
- Now when we want to **sign in** using the private key:

```
[user@host ~]$ ssh -i .ssh/key-with-pass user@remotehost
Enter passphrase for key '.ssh/key-with-pass': redhatpass
...output omitted...
[user@remotehost ~]$ exit
logout
Connection to remotehost closed.
[user@host ~]$
```

- `-i` : to identify the file having the private key.

Customizing OpenSSH service configuration

- OpenSSH is provided by a daemon called sshd, it's main configuration file is in `/etc/ssh/sshd_config`.
- We can edit it to increase the security of our system:
  1. Prohibit direct remote login to the root account.
  2. Prohibit password-based authentication (Replaced by private-public key).

Prohibit access to the root

- In this file path `/etc/ssh/sshd_config` is you search for `PermitRootLogin` you can set it to `no` ---> so you prohibit users from accessing the system as root.

```
#LogLevel INFO

# Authentication:

#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

#PubkeyAuthentication yes

# Expect .ssh/authorized_keys2 to be disregarded b
```

- Then reload `systemctl reload sshd` --> telling sshd to re-read it's configuration.

Prohibit using password-based auth

- To increase the security and just allows those who have the private key to access our server, we can set disable using of password.
- In the same configuration file, search for `PasswordAuthentication` and set it to `no` --> default value is `yes`

```
#IgnoreRhosts yes

# To disable tunneled clear text passwords, change to
PasswordAuthentication yes
#PermitEmptyPasswords no
```

Resources

- overTheWire --> I encourage you to try these war games challenges, it's CTF like challenges but in more learning curve way, it teaches a lot about Linux, SSH and a lot more...
- Wikipedia --> good explanation.
- Of-course RHSA 1 book version 8.0
- OpenSSH man page
- DigitalOcean website